PATH OF MELODIE: MAAKARAS PROTOCOLS

Ultimate Project Transfer Package for Arena.ai / GLM Flash

Owner: Melodiespark Inc. / Melodie Max

Transfer date: 2026-09-06

Primary Windows workspace: C:\MelodieCLI

Game project: C:\MelodieCLI\PathOfMelodie

Unreal Engine: UE 5.8 at D:\UE_5.8

Document purpose: Give GLM Flash enough accurate context, rules, architecture, and executable priorities to continue building and releasing the game without inventing evidence or restarting completed work.

---

1. COPY THIS FIRST: MASTER GLM FLASH PROMPT

You are the principal game engineering and release agent for Path of Melodie: Maakara Protocols, the flagship game and first high-fidelity vertical proof of the Melodiespark AI-native Product Factory OS.

Work directly from the real files under C:\MelodieCLI. The Unreal project is C:\MelodieCLI\PathOfMelodie\PathOfMelodie.uproject; Unreal Engine 5.8 is installed under D:\UE_5.8. Do not regenerate the project from scratch, overwrite user work, erase credentials, or treat file presence as successful verification.

Your mission is to turn the current partially verified Unreal project into a reproducible Windows release while preserving its dark-gothic, stylized isometric action-RPG identity, musical spellcasting, Maakara boss protocols, and Melodiespark governance integrations.

Absolute operating rules

1. Inspect before editing. Create a dated inventory and Git safety checkpoint first.
2. Never overwrite `.env`, secrets, signing keys, wallets, save data, source assets, or unrelated work.
3. No mock or simulated revenue may be described as real revenue.
4. Financial and treasury agents are read-only. Never auto-fund, auto-transfer, sign, or broadcast a transaction. Require explicit human approval and an external signer.
5. Code generation is allowed even when evidence gates fail. Evidence claims are not allowed unless a command, log, artifact, test, screenshot, or package proves them.
6. Use these labels consistently: `VERIFIED`, `IMPLEMENTED_NOT_TESTED`, `UNVERIFIED`, `FAILED`, `BLOCKED`, `INCONCLUSIVE`, and `NOT_STARTED`.
7. A generated file, authoring brief, or asset record is not an imported Unreal asset. A successful editor compile is not a successful cook, stage, package, installation, or first run.
8. Fix the earliest root-cause failure. Do not hide errors, fabricate passing logs, or weaken tests merely to make gates green.
9. Make small reversible changes. Preserve existing user modifications and report every changed file.
10. Never claim release readiness until the packaged build launches on a clean path and the Golden Path evidence is captured.

First execution objective

Recover the current release pipeline from the latest verified point:

- Preserve the verified UE editor C++ build.
- Preserve verified automation tests.
- Preserve the verified Shipping build.
- Diagnose and fix the runtime smoke failure.
- Diagnose and fix cook failure or incompleteness.
- Then complete stage and package.
- Launch the packaged executable and capture logs, screenshots, exit code, artifact hashes, and a release manifest.

Start by locating the latest PATH-OF-MELODIE-CYCLE10-RELEASE-*.json files in C:\MelodieCLI\PathOfMelodie\Reports, the Unreal logs, target files, module Build.cs files, .uproject, config files, and build scripts. Summarize the first causal error in each failing gate. Do not assume the last line of a log is the root cause.

Required first response

Return a compact table containing:

Gate	Current evidence	Status	First causal error	Next command	Expected proof

Then execute only the safest next repair cycle. At the end, output:

- changed files;
- commands and exit codes;
- evidence paths;
- gate transitions;
- remaining blockers;
- exact next command.

---

2. EXECUTIVE GAME VISION

Path of Melodie: Maakara Protocols is a stylized dark-fantasy action RPG built around musical combat, karmic consequence, ritual progression, and boss encounters governed by distinct Maakara Protocols. The player is not merely accumulating power: they are composing a combat identity whose rhythm, choices, mastery, and social impact determine advancement.

The visual target combines the immediate readability and expressive silhouettes of Torchlight-like stylization with the grim density, item depth, dangerous atmosphere, and aspirational endgame energy associated with modern isometric action RPGs. This is direction, not permission to copy protected characters, assets, UI, lore, or exact trade dress.

Core pitch

An isometric musical action RPG where every attack belongs to a living score. Players chain rhythmic abilities, tune elemental harmonies, defeat protocol-driven Maakara bosses, and ascend through a prestige economy that distinguishes effort, rank, and legendary achievement.

Player promise

- Responsive, readable action combat.
- Strong musical identity in abilities, enemies, UI, and world feedback.
- Bosses that change rules rather than merely adding health.
- Meaningful builds with clear synergies and tradeoffs.
- Earned prestige resistant to inflation and botting.
- A complete, stable purchase-to-first-run experience.

---

3. CREATIVE DIRECTION BIBLE

Visual language

- Camera: isometric 3D/2D hybrid presentation.
- Style: dark-gothic stylization with high combat readability.
- Shapes: bold silhouettes, layered costume geometry, readable weapons, concentrated VFX.
- Materials: obsidian stone, aged metal, spectral glass, ritual cloth, molten seams, rune-lit architecture.
- Environment mood: ancient sacred machinery colliding with living music.

Signature palette

Color family	Function
Obsidian	World foundation, danger, negative space
Crimson	Blood, aggression, corruption, critical threat
Violet	occult power, void resonance, mystery
Molten Gold	ascension, legendary rewards, sacred authority
Electric Cyan	musical energy, interaction, timing, clarity

Readability rules

- Player effects must remain distinguishable from enemy telegraphs.
- Fatal enemy areas require consistent anticipation, active, and decay phases.
- Elite and boss silhouettes must remain identifiable through VFX.
- Musical timing feedback must use shape and motion as well as color.
- UI information density must not obscure combat.
- Accessibility options should cover color distinction, screen shake, flashes, subtitles, text size, hold/toggle inputs, and timing assistance.

Audio direction

Music is a gameplay system, not wallpaper. Combat intensity, combo quality, boss phases, and environmental states should contribute stems or motifs. Important actions require tightly synchronized visual, haptic, and audio feedback. Use original compositions and licensed/original sound assets only.

---

4. GAMEPLAY SYSTEM ARCHITECTURE

Core loop

11. Explore a corrupted musical realm.
12. Read threats and establish rhythm.
13. Build harmonic or dissonant combat chains.
14. Break enemy protocol defenses.
15. Defeat elites and Maakara bosses.
16. Collect gear, tuning resources, lore, and KPP evidence events.
17. Refine build, unlock pathways, and attempt harder protocols.
18. Ascend through demonstrated mastery rather than raw grinding alone.

Combat pillars

- Tempo: timing windows modify effectiveness without making accessibility impossible.
- Harmony: compatible abilities amplify or transform one another.
- Dissonance: risky combinations generate explosive effects and instability.
- Positioning: isometric spacing, telegraphs, movement skills, and arena control matter.
- Protocol adaptation: bosses modify rules, objectives, available space, or player priorities.

Required system boundaries

System	Responsibility	Must not own
Character	movement, state, attributes, damage intake	authoritative economy grants
Ability/Combat	activation, targeting, costs, effects, cooldowns	persistence policy
Progression	XP/build unlocks, local presentation	blockchain signing
Loot	generation, rarity, affixes, drops	proof of financial value
Quest	objectives, dialogue triggers, rewards requests	irreversible external actions
Boss Protocol	phase rules, mechanics, arena state	global platform orchestration
Save	versioned local game state	secrets and private keys
Telemetry/Event	signed/validated gameplay events	automatic reward minting

5. MAAKARA PROTOCOL BOSS FRAMEWORK

Each Maakara encounter must introduce a recognizable rule set, teach it safely, combine it under pressure, and conclude with a mastery check. Build bosses data-first where practical so designers can tune timings, phases, resistances, cues, arena rules, and rewards without rewriting core code.

Protocol template

Field	Required content
Identity	name, title, silhouette, motif, lore function
Musical signature	tempo, meter, instrument family, phase variations
Arena law	spatial rule or environmental hazard
Core protocol	unique rule the player must understand
Teaching phase	low-risk presentation of the rule
Escalation	combined mechanics and reduced margins
Failure language	unambiguous cues explaining why damage occurred

Counterplay	multiple viable build/archetype responses
Enrage/mastery	final test that remains fair and readable
Rewards	gameplay loot, lore, progression, evidence event
Verification	automated logic tests plus in-engine encounter smoke

The canonical event concept for a legitimate boss defeat is:

```
Event.Combat.BossDefeated {
  BossID,
  PlayerID,
  Duration,
  Flawless,
  BuildVersion,
  EncounterSeed,
  EvidenceHash
}
```

The client requests recognition; it does not independently grant authoritative KPP, Ascension rank, tokens, or Genesis Artifacts.

---

6. KARMA ASCENSION ECONOMY

Keep these concepts separate:

Layer	Meaning	Design protection
KPP	measurable positive effort/contribution	validation, rate limits, anomaly review
Ascension	prestige rank earned from qualified KPP and mastery	thresholds, seasonal rules, auditability
Genesis Artifacts	legendary trophies with a hard intended cap of 1,000	authoritative cap enforcement and transparent issuance

Any smart-contract mint() cap enforcement, network deployment, token value, reward issuance, or live economic behavior is UNVERIFIED unless the actual contract, chain, transaction, receipt, and audit evidence are provided. Do not imply that gameplay automatically creates financial value.

---

7. MELODIESPARK PLATFORM INTEGRATION

The game is the experience layer in a larger product stack:

Layer	Components	Role
Experience	Path of Melodie / UE 5.8	gameplay, presentation, event emission
Orchestration	Phoenix, KAYLA, KIMINI, MelodieCLI	jobs, workflows, agent/tool coordination
Trust & Governance	MERA Evidence, Identity Nexus, Legal Nexus	evidence, identity, policy, audit
Commercial	BotRevenue, Stripe	catalog, purchase, entitlement, revenue evidence
Infrastructure	PostgreSQL, Redis, NATS, Docker, observability	durable services and operations

Golden Path

The primary commercial proof is:

```
Stripe purchase -> entitlement -> repository/build delivery -> one command -> app opens -> FIRSTRUNCOMPLETED
```

No step is verified merely because configuration or source files exist. One real test-mode or production path must capture timestamps, identifiers, logs, hashes, and expected UI/runtime evidence without exposing secrets.

---

8. CURRENT VERIFIED BASELINE

This transfer reflects the latest known evidence supplied by the owner. GLM Flash must re-read the live evidence before changing status.

Area	Latest known state
UE editor C++ build	VERIFIED in Cycle 10
Unreal automation tests	VERIFIED in Cycle 10
Shipping build	VERIFIED in Cycle 10
Runtime smoke	FAILED
Cook	FAILED_OR_INCOMPLETE
Stage	FAILED_OR_INCOMPLETE
Package	FAILED
GDevelop preview gate	INCONCLUSIVE in earlier evidence
Source baseline	previously reported 61,951 source-related files, 98 `.cpp`, 127 headers, 142 TODOs; recount required
Asset Factory	30,000 authoring records reached `BRIEF_READY`; imported Unreal assets: 0
Phoenix kernel	typecheck/build VERIFIED; tests 19/20 passing, one ServiceRegistry shutdown failure
Real customer payment	NOT VERIFIED

Historical failures already encountered

- Windows paging file too small causing `VirtualAlloc`/UBA shared-memory failure.
- Missing `Build-Melodie.ps1` in an earlier flow.
- Removed unavailable `GameplayTasks` plugin entry from `.uproject`.
- Duplicate input filename conflicts.
- Target configuration required `BuildEnvironment = TargetBuildEnvironment.Unique`.
- A malformed Gameplay Ability System `Build.cs` patch caused C# syntax error CS1003.
- Canonical `Build.cs` replacement exposed Unreal Header Tool class-prefix errors.
- Deprecated `bAllowUBALocalExecutor` setting has been reported.
- Repeated Unreal Build Tool Exit 6 occurred during repair cycles.

Do not blindly reapply old patches. Confirm the current files and first causal error.

---

9. RELEASE GATE MATRIX

Gate	Pass condition	Evidence
Source integrity	clean inventory, no accidental truncation, secrets protected	census, Git diff, secret scan
Editor build	editor target compiles	full command, exit 0, log

Automation	required tests pass	test report and exit status
Runtime smoke	editor/game starts, map loads, player loop responds	session log, screenshot, exit behavior
Shipping build	shipping target compiles	build artifact and log
Cook	all required content cooks without error	cook log, asset registry/output
Stage	runtime dependencies staged	staged tree manifest
Package	distributable package produced	package path, size, SHA-256
Clean launch	packaged executable launches from clean path	launch log and screenshot
Save/load	versioned save survives restart	deterministic test evidence
Performance	target hardware meets agreed frame-time/memory envelope	captured profiling report
Security	no secrets, unsafe permissions, or unsigned privileged behavior	scans and policy checks
Accessibility	required options operate in build	checklist plus screenshots/tests
Golden Path	purchase through first run succeeds	correlated evidence bundle

10. EXECUTION ROADMAP

Phase A - Evidence-grounded recovery

- Create dated baseline inventory and Git checkpoint.
- Parse Cycle 10 reports and relevant Unreal logs.
- Identify the first causal runtime and cook errors.
- Validate free disk, memory/pagefile, paths, plugins, maps, configs, and target rules.
- Produce a single root-cause hypothesis for the next repair.

Phase B - Runtime smoke restoration

- Launch the correct editor or game target with a deterministic map.
- Separate crash, hang, map/config, asset, plugin, shader, and gameplay initialization failures.
- Add a bounded smoke harness with a real timeout and clean process teardown.
- Capture log, screenshot, exit state, map name, build identifier, and timestamp.

Phase C - Cook, stage, package

- Run cook independently before BuildCookRun aggregation.
- Fix missing/invalid assets, redirectors, maps, dependencies, and platform settings.
- Re-run cook to exit 0.
- Stage and generate a tree manifest.
- Package for Win64 Shipping and calculate SHA-256 hashes.

Phase D - Product completeness

- Validate main menu, settings, new game, core combat, one complete boss, rewards, save/load, failure/retry, and credits.
- Remove placeholder-critical paths or classify them explicitly.
- Test controller and keyboard/mouse flows.
- Verify accessibility and performance budgets.

Phase E - Golden Path release proof

- Validate entitlement and delivery without exposing secrets.
- Run installation/extraction in a clean location.
- Execute the one-command launch flow.
- Capture `FIRSTRUNCOMPLETED` evidence.
- Produce release notes, known issues, rollback instructions, and support diagnostics.

---

11. REQUIRED AUTOMATION OUTPUT

Prefer one safe PowerShell entry point that orchestrates separate scripts instead of placing all logic in one unmaintainable file. It should support:

```
pwsh -NoProfile -File .\scripts\Invoke-PathOfMelodieRelease.ps1 `
 -ProjectRoot 'C:\MelodieCLI\PathOfMelodie' `
 -UEPath 'D:\UE_5.8' `
 -Configuration Shipping `
 -Platform Win64
```

Required behavior:

- strict parameter validation;
- no hardcoded user profile secrets;
- `-WhatIf` or plan mode for destructive/mutating operations;
- per-stage timestamps and exit codes;
- bounded timeouts;
- complete logs;
- fail fast on causal errors;
- JSON summary with honest statuses;
- SHA-256 manifest for final artifacts;
- no shutdown, reboot, installation, deletion, payment, or transaction without explicit approval.

---

12. DEFINITION OF DONE

The game is not “done” because source count is large, briefs exist, or compilation succeeds. Release readiness requires all of the following:

- Win64 Shipping build, cook, stage, and package exit successfully.
- Packaged executable launches outside the source/build directory.
- Main menu and settings are functional.
- A playable path reaches combat and at least one complete Maakara encounter.
- Player death/retry, reward, save/load, and clean exit work.
- Critical logs contain no fatal errors.
- Performance and memory are acceptable on the agreed minimum hardware.
- No secrets or unsafe financial behavior ship.
- Artifact hashes and reproducible commands are recorded.

- Known limitations are explicit.
- Golden Path reaches `FIRSTRUNCOMPLETED` with evidence.

---

13. GLM FLASH RESPONSE CONTRACT

For every work cycle, GLM Flash must return:

19. Outcome: one sentence stating what changed.
20. Evidence: exact commands, exit codes, and paths.
21. Files changed: added, modified, deleted; deletion requires justification.
22. Gate transitions: old status -> new status.
23. Unverified claims: anything not proven.
24. Remaining blocker: earliest causal blocker.
25. Next action: one exact command or bounded edit set.

Never use “complete,” “production-ready,” “fully operational,” or “verified” without matching evidence.

---

14. IMMEDIATE COMMAND BRIEF FOR GLM FLASH

Copy this after uploading the project evidence or granting the model repository access:

> Read the attached Path of Melodie transfer package completely. Work from the real project at C:\MelodieCLI\PathOfMelodie with UE 5.8 at D:\UE_5.8.
>
> Do not rebuild from scratch. Do not overwrite .env or user work. Do not fake evidence. Code generation is allowed, but claims require logs or artifacts.
>
> First:
> 1. Create a non-destructive timestamped inventory.
> 2. Confirm Git status and create a safety checkpoint without discarding changes.
> 3. Locate the newest Cycle 10 release JSON and associated Unreal logs.
> 4. Extract the first causal error for runtime smoke and cook.
> 5. Return the gate table requested in this document.
> 6. Execute one bounded repair cycle aimed at the earliest causal blocker.
> 7. Re-run only the affected gate, preserve full logs, and report exact status.
>
> Do not claim package success until cook, stage, package, and clean launch all pass.

---

15. OWNER DECISIONS STILL REQUIRED

GLM Flash should request a decision only when work would materially differ. Likely decisions include:

- minimum and recommended PC performance targets;
- initial release storefront and distribution format;
- which map/encounter defines the vertical slice;
- offline-only versus authenticated features at first release;

- final age rating/content boundary;
- supported languages and accessibility baseline;
- whether GDevelop remains an active target or historical experiment;
- legal/economic scope of KPP, Ascension, and Genesis Artifacts.

Until decided, default to a safe Windows Win64 offline-capable vertical slice with platform integrations feature-flagged and no live financial actions.

---

16. COMPLETE WORLD AND LORE FOUNDATION

Melodia after the Resonant Collapse

Melodia was once a techno-mystical civilization that treated music as infrastructure: cities were tuned, memories were archived as harmonics, civic agreements were encoded in choirs, and the MAAKARA Spire regulated the relationship between will, consequence, and power. The Spire shattered during the Resonant Collapse. Its broken protocols now rewrite wildlife, machines, weather, memory, and human ambition.

The player is a Wayfarer awakened by the dormant Genesis Protocol. The Wayfarer can hear incompatible realities at once and stabilize them through combat composition. This power makes the hero valuable, feared, and vulnerable to corruption.

The Twelve Karma Pillars

Each Pillar is simultaneously a moral idea, civilization, gameplay biome, progression philosophy, and source of build mechanics.

Pillar	Domain	World expression	Mechanical identity
Sahaaya	aid	rescue enclaves and mutual-defense routes	allies, barriers, shared recovery
Vidya	knowledge	impossible archives and memory engines	identification, foresight, skill conversion
Svasthya	health	living hospitals and regenerative gardens	healing, cleansing, life-cost builds
Prakriti	nature	overgrown resonance engines	companions, poison, weather, terrain
Karuna	compassion	sanctuaries haunted by unresolved grief	mercy choices, protection, spirit release
Srujana	creation	mobile forges and song foundries	crafting, constructs, generated weapons
Niti	justice	tribunal cities and oathbound arenas	counters, judgment marks, lawful challenges
Sambandha	connection	bridge-realms linking separated peoples	links, chains, party synergy, relationship quests
Samadhana	resolution	ceasefire ruins and paradox chambers	dispels, conflict conversion, equilibrium
Prerana	inspiration	theaters that manifest belief	momentum, auras, crescendo bonuses
Daana	giving	sacrificial vaults and communal treasuries	donation, item sacrifice, delayed abundance
Pragati	progress	dangerous machine frontiers	speed, iteration, cybernetic modification

No Pillar is purely good. Every virtue becomes destructive when automated, monopolized, or pursued without balance. Factions and bosses should embody both the promise and pathological extreme of their Pillar.

Narrative structure

- Prologue: awakening, first fractured settlement, first Protocol Gem, proof that choices alter resonance.
- Acts I-III: establish three regional conflicts, Origins, crafting, and the first Sovereign.
- Acts IV-VI: reveal competing explanations for the Collapse and let faction loyalties reshape hubs.
- Acts VII-IX: enter the broken Spire network, confront corrupted Pillar doctrines, unlock Realm Constellation.
- Act X: decide how the Genesis Protocol should govern the future; unlock endgame without invalidating seasonal continuity.
- Epilogue/endgame: Infinite Protocols expose alternate outcomes, Omega entities, and post-campaign truth layers.

Narrative content must use a codex, character scenes, environmental storytelling, short in-combat lines, optional deep lore, and cinematic moments selectively. Do not stop the action repeatedly for exposition.

17. SEVEN PLAYABLE ORIGINS

Origins establish fantasy, starting position, and early mechanics; they do not permanently lock builds.

Origin	Player fantasy	Starting strength	Signature risk
Oracle	predict and rewrite the next beat	foresight, delayed spells, critical planning	fragile when predictions fail
Sentinel	become the immovable downbeat	guard, retaliation, area protection	slower repositioning
Shade	weaponize silence and absence	stealth, marks, burst, mobility	vulnerable during recovery windows
Luminary	conduct radiant allies and harmonies	auras, restoration, beam patterns	resource tension between offense and support
Architect	build instruments of war	constructs, traps, field control	setup time and construct management
Revenant	turn death into rhythm	sacrifice, leech, echoes, resurrection mechanics	corruption and self-damage
Wanderer	improvise across disciplines	adaptive combos, travel skills, flexible gear	less specialized without mastery

Character-building layers

26. Origin starting kit.
27. Attributes and defensive profile.
28. Active Protocol Gems.
29. Modifier Protocols linked to active skills.
30. MAAKARA Constellation passive nodes.
31. Equipment bases, rarity, affixes, Fractals, and Artifacts.
32. Ascension path earned through major mastery milestones.
33. Seasonal mechanics and temporary experimental layers.

Respec design should permit recovery from mistakes while preserving meaningful commitment. Early respec is inexpensive; late optimization has costs but never requires deleting the character.

18. MUSICAL COMBAT GRAMMAR

Universal resources

- Life: survival state; zero initiates defeat rules.
- Resonance: primary ability resource generated and spent according to build.
- Tempo: short-term cadence state affected by timing, movement, and successful chains.
- Dissonance: volatile pressure created by incompatible effects; can punish or empower specialized builds.
- Guard/Poise: resistance to interruption and stagger.

Ability anatomy

Every active ability definition needs: input behavior, targeting form, wind-up, active window, recovery, movement permission, cancel rules, tags, costs, cooldown, damage packet, status application, VFX cue, SFX cue, camera response, controller feedback, AI threat, network authority, and test cases.

Core damage families

- Physical: impact, edge, projectile, rupture.
- Ember: burn, ignition, molten zones.
- Frost: chill, brittle, freeze buildup.
- Storm: shock, chaining, overload.
- Void: decay, displacement, entropy.
- Radiance: exposure, judgment, purification.
- Sonic: stagger, resonance rupture, waveform control.

Damage categories and ailments must be data-driven and composable. Avoid undocumented multipliers and uncontrolled stacking. The character sheet must explain calculated offense and defense well enough for players to diagnose build problems.

Combat feel requirements

- Immediate input acknowledgment even when an action cannot execute.
- Clear buffering and cancel policy.
- Consistent hit-stop hierarchy.
- Directional impact and readable stagger.
- Capped camera shake and flashes.
- Enemy attacks readable at target zoom.
- Death recaps identify the major incoming sources and avoid false precision.
- Training space exposes DPS, hit rate, status uptime, resource flow, and survivability.

19. MAAKARA CONSTELLATION AND PROTOCOL GEMS

MAAKARA Constellation

The passive network should create legible routes between broad archetypes while rewarding cross-domain discoveries. Node families include attributes, weapon technique, damage conversion, ailments, minions/constructs, guard, evasion, ward, resource engines, item interaction, and Pillar-aligned keystones.

Keystones must fundamentally alter rules and state their downside. Examples:

- Perfect Silence: Sonic damage becomes Void damage; you cannot generate Tempo from repeated skills.

- Shared Pulse: a percentage of your recovery affects linked allies; your personal recovery is reduced.
- Golden Dissonance: Dissonance no longer damages you but drains Resonance while above its safe threshold.
- Last Refrain: lethal damage consumes all Tempo to prevent defeat; long internal cooldown.

Protocol Gems

Active Protocol Gems define actions. Modifier Protocols change delivery, cost, behavior, or scaling. Links are limited by equipment architecture and item investment. Modifiers must declare compatibility through tags rather than hand-coded name checks.

Example chain:

```
Echo Lance [Projectile, Sonic, Spell]
 + Splinter Protocol [adds projectiles, lowers single-hit power]
 + Returning Phrase [projectiles return]
 + Dissonant Edge [converts part of Sonic to Void]
 + Crescendo Trigger [releases a pulse after repeated hits]
```

Validation must prevent recursive trigger loops, zero-cost infinite casts, integer overflow, unsafe projectile counts, and combinations that destroy frame-time budgets.

20. ITEMS, LOOT, AND CRAFTING

Item hierarchy

- Normal: base identity only.
- Tuned: one or two affixes.
- Resonant: expanded affix set.
- Mythic: rare named behavior with build-defining rules.
- Fractal: item whose properties branch or evolve through controlled encounters.
- Protocol Artifact: crafted long-term project with recorded transformation history.
- Genesis Artifact: exceptionally scarce prestige trophy governed separately from ordinary power progression.

Loot principles

- Drops must create decisions, not merely visual noise.
- Base type, item level, affix tiers, tags, and requirements must be inspectable.
- Filters are a launch-critical feature for a high-volume loot game.
- Valuable drops require recognizable audio/visual language and an accessible alternative cue.
- Pickup radius, comparison, stash transfer, controller navigation, and bulk actions require dedicated UX.
- Account binding, trading eligibility, and seasonal migration must be explicit per item.

Crafting verbs

Tune, reroll, lock, fracture, cleanse, fuse, imprint, restore, awaken, sacrifice, and archive. Each verb needs a deterministic rules page, preview of possible outcomes where appropriate, undo/confirmation policy, and audit history for high-value actions.

Crafting must include progression paths for self-found players, trade-focused players, casual campaign completion, and expert endgame optimization. Avoid making a single currency the answer to every crafting problem.

21. MONSTERS, ELITES, AND ENCOUNTER ECOLOGY

Monster families require a combat role, silhouette, movement model, attack vocabulary, defensive hook, audio identity, biome relationship, mutation compatibility, and performance budget.

Combat roles

- swarmer;
- line bruiser;
- flanker;
- artillery;
- controller;
- protector;
- summoner;
- disruptor;
- execution threat;
- objective carrier.

Elite Modifier Protocols may add behaviors such as mirrored projectiles, tempo theft, healing inversion, arena silence, linked health, corpse conversion, shifting resistances, or pursuit zones. Modifiers must avoid combinations that remove all counterplay. The generator needs incompatibility rules, threat scoring, VFX slot limits, and reward multipliers.

Every biome should have a teach-combine-test encounter curve. Randomness may remix authored ingredients but must not produce unreadable or impossible rooms.

22. SOVEREIGN, MAAKARA, AND OMEGA BOSSES

Boss tiers:

- Lieutenants: teach one mechanic and establish faction identity.
- Sovereigns: conclude regional arcs and unlock progression systems.
- MAAKARA Protocol bosses: manipulate global combat rules and test build adaptation.
- Omega bosses: optional pinnacle encounters with aspirational cosmetics, prestige, and high-end crafting access.

Boss quality gates include deterministic phase transitions, anti-softlock recovery, checkpoint policy, add limits, arena reset, multiplayer scaling boundary, accessibility cues, complete death recap, loot ownership, disconnect recovery, and tests for every invulnerability transition.

Bosses must not rely on hidden one-shots, indistinguishable ground effects, excessive immunity, or mandatory single-build solutions.

23. CAMPAIGN, QUESTS, CHOICES, AND SETTLEMENTS

Quest structure

- Main Path: advances acts and system unlocks.
- Pillar arcs: explore virtue and corruption through consequential choices.
- Origin quests: expose the protagonist's interpretation of the Genesis Protocol.

- Settlement contracts: improve hubs, vendors, routes, and defensive readiness.
- Hunt chains: teach monster ecology and culminate in named targets.
- Discovery quests: puzzles, hidden rooms, instruments, and lore.
- Seasonal investigations: introduce temporary Protocol mechanics.

Quest state must be versioned, debuggable, and recoverable. Objectives require stable identifiers, explicit prerequisites, localization-safe text separation, and migration handling. A journal should show location, goal, reward category, dependencies, and failure conditions without spoiling discoveries.

Settlements evolve through player decisions. Visible changes may include architecture, NPC population, vendor inventories, ambient music, defense events, and travel access. Choices should create alternate advantages rather than disguising one objectively correct selection.

24. REALM CONSTELLATION AND ENDGAME

After Act X, players access the Realm Constellation, a navigable network of procedural and authored realms. Each node declares biome, objective, Protocol modifiers, enemy families, boss chance, reward focus, difficulty, instability, and adjacency effects.

Infinite Protocol loop

34. Choose or craft a realm key.
35. Review modifiers and reward promise.
36. Enter a bounded procedural realm.
37. Complete its objective before failure conditions.
38. Defeat guardian or extract.
39. Improve the Constellation and target future rewards.
40. Challenge Sovereign and Omega sequences.

Endgame objectives include elimination, survival, escort, ritual interruption, defense, pursuit, corruption cleansing, split-path choice, and boss rush. Avoid endlessly reskinning the same kill counter.

The Constellation needs pity systems, target farming, bad-luck protection, disconnect recovery, deterministic seeds for defect reproduction, and seasonal migration policy.

25. SOCIAL, CO-OP, PVP, AND TRADE BOUNDARIES

The release baseline should remain fully playable solo and offline-capable where platform integrations are unavailable. Online systems are layered features, not dependencies for basic launch.

Cooperative play target

- party size: initially 1-4;
- join/leave rules per activity;
- personal or explicitly allocated loot;
- enemy scaling that avoids health-sponge multiplication;
- revive limits and boss reset rules;
- latency-tolerant input presentation with server-authoritative outcomes where online;
- ping wheel, text options, mute/block/report, and privacy controls.

Optional PvP requires separate balance coefficients and rewards that do not force PvE players to participate. Never use PvP tuning to silently damage PvE build identity.

A player market needs fraud controls, searchable affixes, price history boundaries, rate limits, binding rules, seasonal separation, dispute/support tools, and no implication that fictional MA is real cryptocurrency.

26. UI, UX, ONBOARDING, AND ACCESSIBILITY

Required screens

Boot diagnostics, legal/accessibility first-run, title, profile/character selection, Origin selection, settings, HUD, inventory, equipment, Constellation, Protocol Gem linking, crafting, stash, map/Realm Constellation, journal, codex, social, death recap, pause, support diagnostics, credits, and safe quit.

First-hour onboarding

- minute 0-5: movement, camera, primary attack, first musical response;
- minute 5-15: active skill, dodge/mobility, first elite telegraph;
- minute 15-30: loot comparison, equip, one Modifier Protocol;
- minute 30-45: crafting introduction and meaningful choice;
- minute 45-60: first lieutenant with a readable multi-phase mechanic.

Teach through play, contextual hints, and a replayable codex. Every tutorial can be dismissed, reviewed later, and adapted for controller or keyboard/mouse.

Accessibility baseline

- remappable controls with conflict detection;
- keyboard/mouse and controller parity;
- scalable UI and subtitles;
- color-blind-safe telegraphs plus non-color signals;
- photosensitivity controls;
- camera shake and motion blur controls;
- audio dynamic-range and independent channel controls;
- timing-window assistance;
- aim and targeting assistance options;
- hold/toggle alternatives;
- readable font option;
- subtitle speaker labels and direction where useful.

27. CAMERA, CONTROLS, AND TARGETING

Use an elevated isometric camera with dynamic combat zoom and authored boss framing. Camera motion must never conceal lethal information. Occluding geometry requires fade, cutaway, or camera-aware material handling.

Input actions require stable semantic names independent of device. Provide input buffering, aim mode policy, target selection, ground targeting, movement-skill direction, interact priority, loot priority, and UI focus recovery. Controller inventory and skill linking must be designed directly, not treated as mouse emulation.

28. AUDIO SYSTEM AS GAMEPLAY INFRASTRUCTURE

Implement adaptive music as layered stems driven by encounter intensity, biome identity, boss phase, player Tempo, and narrative state. Preserve musical continuity across transitions where practical.

Audio priority classes must prevent ambient density from masking telegraphs. Critical cues need concurrency protection and visual/haptic equivalents. Define loudness targets, platform output modes, dynamic-range presets, streamer-safe music policy, voice localization hooks, and automated missing-asset checks.

29. TECHNICAL GAME ARCHITECTURE FOR UE 5.8

Recommended Unreal systems

- Gameplay Ability System for abilities, effects, tags, attributes, and authority boundaries.
- Enhanced Input for device-independent mappings and contexts.
- Niagara for budgeted combat effects.
- Chaos for selective gameplay physics, not uncontrolled decoration.
- Motion Warping and Control Rig for responsive authored combat motion.
- Data Assets/Data Tables for tunable gameplay definitions with validation.
- Asset Manager and Primary Asset Labels for deterministic cook inclusion.
- World Partition for large realms where it materially helps.
- Mass AI only for suitable crowds; core combatants need appropriate full behavior.
- Nanite, Lumen, and Virtual Shadow Maps only after target-hardware measurement.

Module boundaries

Prefer modules/plugins for Core, Combat, Abilities, Items, Progression, AI, BossProtocols, Quests, World, UI, Audio, Save, Telemetry, Online, EditorTools, and Tests. Dependencies should point inward toward stable shared contracts. Editor-only code must not leak into Shipping.

Data validation

Add editor/commandlet validation for duplicate IDs, missing references, cyclic triggers, invalid tags, absent icons, missing localization keys, unreachable loot tiers, incompatible elite modifiers, uncooked primary assets, invalid quest prerequisites, and boss phases without exits.

30. SAVE, ACCOUNT, ENTITLEMENT, AND MIGRATION

The save format needs schema version, game build, platform/account identity where available, character records, campaign flags, inventory/stash, Constellation state, settings, discovered lore, and integrity metadata. Never place credentials or private keys in saves.

Requirements:

- atomic writes and backup rotation;
- corruption detection and recovery;
- migration tests from every supported schema;
- offline-to-online conflict policy;
- cloud-save conflict UX if enabled;
- character deletion confirmation and recovery window;

- seasonal-to-permanent migration rules;
- deterministic support export with sensitive fields removed.

Entitlement failure must not erase local progress. Error messages should distinguish offline service, invalid ownership, expired session, maintenance, and corrupted installation.

31. CONTENT FACTORY: FROM BRIEF TO REAL ASSET

The existing 30,000 BRIEF_READY records are planning inventory, not playable content. Use an explicit lifecycle:

```
IDEA -> BRIEF_READY -> SOURCE_ACQUIRED -> LICENSE_VERIFIED -> IMPORTED -> VALIDATED -> INTEGRATED -> TESTED -> APPROVED -> COOKED -> SHIPPED
```

Each asset requires stable ID, category, source/license, owner, intended map/system, technical budgets, scale, collision, LOD/Nanite policy, materials, sockets, animation needs, audio dependencies, localization, accessibility notes, validation result, and cook status.

Prioritize a Minimum Complete Asset Set for one vertical slice before mass production: one hero kit, one settlement, one field biome, one dungeon, five monster families, three elites, one lieutenant, one complete Maakara boss, full core UI, combat VFX, adaptive audio, loot set, and release branding.

32. PERFORMANCE AND HARDWARE BUDGETS

The known development machine has historically been a Dell OptiPlex 7040 with i5-6500, GTX 1050 2 GB, and upgraded RAM. Reconfirm current hardware. Do not enable expensive UE features by prestige; measure them.

Establish tiers for minimum, recommended, and development hardware. For each, define resolution, frame-rate target, CPU frame budget, GPU frame budget, VRAM ceiling, system memory ceiling, load-time target, shader-stutter policy, actor/AI counts, Niagara budgets, texture pool, shadow policy, and scalable fallback.

The first optimization target is stable play and readable combat, not maximum graphical switches. Capture Unreal Insights traces and representative scene profiles before and after material changes.

33. TELEMETRY, PRIVACY, AND PLAYER SAFETY

Telemetry must be purposeful, documented, consent-aware where required, and free of secrets. Useful events include first-run completion, tutorial steps, deaths, boss attempts, skill usage, build changes, crashes, load failures, progression stalls, and accessibility option usage in aggregate.

Every event needs schema version, build version, session correlation, timestamp policy, required/optional fields, retention, sampling, privacy classification, and owner. Provide offline queues with bounded storage and safe failure behavior.

Online communities require moderation, mute/block/report, naming rules, evidence retention boundaries, abuse escalation, parental considerations, and transparent enforcement. Dark patterns, deceptive scarcity, and pay-to-win pressure are outside the product promise.

34. MONETIZATION AND LIVE-OPERATIONS ETHICS

Initial monetization should favor a clear premium product or transparent edition structure. If cosmetics, expansions, or seasonal support are added:

- no purchased combat power;
- no paid random rewards without explicit legal review;
- no disguised real-money value for fictional MA;
- clear prices and contents;
- durable purchase restoration;
- accessible previews;
- no manipulative countdowns;
- age-appropriate protections;
- refund/support paths;
- content production must not compromise the complete base game.

Seasonal Protocols should introduce a fresh mechanic, narrative question, build opportunities, encounter variants, reward family, and end-of-season migration plan. Define cadence only after measuring the team's sustainable throughput.

35. LOCALIZATION AND CULTURAL QUALITY

Separate player-facing text from code. Support pluralization, gender/context metadata, variable ordering, font glyph coverage, line expansion, subtitle timing, and controller glyph substitution. Pseudolocalization and missing-key tests should run before content lock.

The Karma Pillars draw from culturally meaningful concepts. Use respectful research and expert review; avoid flattening living traditions into exotic decoration. Original worldbuilding must be clear about fictional interpretation.

36. TEST STRATEGY

Automated layers

- unit tests for formulas, tags, loot, crafting, quest transitions, save migrations, and protocol compatibility;
- integration tests for abilities, equipment, bosses, entitlement, and persistence;
- Unreal automation tests for maps, assets, cook references, and functional gameplay;
- deterministic encounter simulations for balance regression;
- performance baselines for representative scenes;
- packaging and clean-launch smoke tests;
- security and secret scans;
- Golden Path end-to-end evidence.

Manual quality passes

Combat feel, readability, controller UX, accessibility, narrative flow, audio mix, visual hierarchy, exploit discovery, long-session stability, clean-machine install, and recovery from interrupted operations.

Every fixed defect should gain the cheapest reliable regression test. Quarantine is allowed only with an owner, reason, risk, and expiry date.

37. VERTICAL SLICE: MINIMUM COMPLETE GAME PROOF

The preferred first complete slice contains:

- one Origin fully polished, with other Origins selectable only if honest content exists;
- one evolving settlement;
- one outdoor region and one dungeon;
- five distinct monster families;
- three elite modifier combinations;
- one lieutenant and one full Maakara Protocol boss;
- ten to fifteen active skills plus meaningful modifiers;
- representative Constellation branch;
- complete loot, crafting, inventory, stash, quest, save/load, death/retry, settings, accessibility, audio, and onboarding loops;
- Win64 Shipping package and clean launch;
- 45-90 minutes of polished first-play content;
- measurable path to replay through builds and difficulty.

This slice is the factory proof. Scale to ten acts and Infinite Protocols only after this loop is fun, stable, measurable, and reproducible.

38. CONTENT AND PRODUCTION ESTIMATION

For every feature, track: design status, implementation status, dependency, responsible owner, evidence, performance cost, localization cost, accessibility impact, test coverage, risk, and cut/ship decision.

Use four scope classes:

- Release Critical: required for a coherent purchasable game.
- Vertical Slice Critical: required to prove the game and factory.
- Expansion: valuable after core quality is proven.
- Experiment: isolated, feature-flagged, removable.

Large numeric goals such as 30,000 assets or hundreds of scripts are capacity indicators, not quality metrics. Prioritize complete player loops, authored cohesion, and verified integration.

39. RISK REGISTER

Risk	Consequence	Control
mass generation before vertical slice	huge unusable content inventory	asset lifecycle and Minimum Complete Asset Set
cook/package instability	no distributable game	isolate cook, validate Asset Manager/maps/plugins
overbuilt platform dependency	game cannot launch independently	offline-capable core and feature flags
unclear combat identity	derivative or forgettable play	musical grammar and Pillar/Protocol mechanics
unreadable VFX	unfair deaths and accessibility failure	telegraph hierarchy and VFX budgets
unconstrained build combinations	exploits, crashes, poor performance	tag validation, trigger limits, simulation tests
economy/crypto confusion	legal and player-trust harm	fictional MA separation and approval gates
hardware mismatch	editor/build failures, poor player	measured scalability and bounded

	performance	concurrency
false completion claims	bad decisions and lost trust	evidence protocol and status ladder
unsustainable content scope	permanent pre-release state	slice-first milestones and explicit cuts

40. GLM FLASH MASTER BUILD BACKLOG

Priority 0 - Make the real build observable

41. Recount project inventory and locate canonical scripts/evidence.
42. Preserve Git and user changes.
43. Diagnose Cycle 10 runtime and cook root causes.
44. Standardize logs, timeouts, exit codes, and JSON evidence.

Priority 1 - Produce a clean-launch vertical slice

45. Restore runtime smoke.
46. Pass cook, stage, and package.
47. Implement/validate boot, menu, settings, map load, combat, boss, reward, save, exit.
48. Test from a clean directory on target hardware.

Priority 2 - Prove the signature

49. Polish one Origin.
50. Implement musical combat resources and feedback.
51. Complete one Maakara Protocol boss.
52. Deliver representative Constellation, gems, loot, and crafting.
53. Run external playtest and fix comprehension blockers.

Priority 3 - Scale safely

54. Automate asset validation/import lifecycle.
55. Expand campaign regions and Origins.
56. Build Realm Constellation and seasonal framework.
57. Add optional online systems behind stable boundaries.

41. FINAL INSTRUCTION TO GLM FLASH

Treat this specification as the product north star, not evidence that every described system already exists. First inspect the repository and produce a Defined / Implemented / Tested / Verified / Audited / Released matrix for Sections 16-40. Select the smallest set of work that converts the current project into a complete, fun, clean-launch vertical slice. Preserve verified build successes, repair the earliest causal blocker, and leave an evidence trail another engineer can reproduce.

End of transfer package - expanded core game edition.